

DBMS SKILL TASK – WEEK 6

Product Review and Rating Management System

1. Introduction

The **Product Review and Rating Management System** is developed for the Vilvah e-commerce database to manage customer feedback and product ratings. The system stores reviews and ratings given by customers for products. It helps retrieve product review details, calculate average product ratings using aggregate functions, and identify highly rated products.

2. Objectives

The main objectives of this task are:

- To create Review and Rating tables.
- To store customer feedback and product ratings.
- To retrieve product review details.
- To calculate average product ratings.
- To identify highly rated products.
- To use SQL aggregate functions such as AVG() and GROUP BY.

3. Database Tables

3.1 Review Table

Attribute	Data Type	Key
Review_ID	NUMBER(10)	Primary Key
Customer_ID	NUMBER(10)	Foreign Key
Product_ID	NUMBER(10)	Foreign Key
Review_Text	VARCHAR2(500)	—
Review_Date	DATE	—

3.2 Rating Table

Attribute	Data Type	Key
Rating_ID	NUMBER(10)	Primary Key
Customer_ID	NUMBER(10)	Foreign Key
Product_ID	NUMBER(10)	Foreign Key
Rating_Value	NUMBER(1)	—
Rating_Date	DATE	—

4. Table Creation

4.1 Review Table

```
SQL> CREATE TABLE Review (  
 2     Review_ID NUMBER(10) PRIMARY KEY,  
 3     Customer_ID NUMBER(10) NOT NULL,  
 4     Product_ID NUMBER(10) NOT NULL,  
 5     Review_Text VARCHAR2(500),  
 6     Review_Date DATE,  
 7     CONSTRAINT FK_REVIEW_CUSTOMER  
 8         FOREIGN KEY (Customer_ID)  
 9         REFERENCES Customer(Customer_ID),  
10     CONSTRAINT FK_REVIEW_PRODUCT  
11         FOREIGN KEY (Product_ID)  
12         REFERENCES Product(Product_ID)  
13 );  
  
Table created.
```

4.2 Rating Table

```
SQL> CREATE TABLE Rating (  
 2     Rating_ID NUMBER(10) PRIMARY KEY,  
 3     Customer_ID NUMBER(10) NOT NULL,  
 4     Product_ID NUMBER(10) NOT NULL,  
 5     Rating_Value NUMBER(1) NOT NULL,  
 6     Rating_Date DATE,  
 7     CONSTRAINT FK_RATING_CUSTOMER  
 8         FOREIGN KEY (Customer_ID)  
 9         REFERENCES Customer(Customer_ID),  
10     CONSTRAINT FK_RATING_PRODUCT  
11         FOREIGN KEY (Product_ID)  
12         REFERENCES Product(Product_ID),  
13     CONSTRAINT CHK_RATING_VALUE  
14         CHECK (Rating_Value BETWEEN 1 AND 5)  
15 );  
  
Table created.
```

5. Inserting Review Details

```
SQL> INSERT INTO Review
  2 VALUES (1, 1, 1001, 'Very good product and suitable for my skin.', DATE '2026-09-20');
1 row created.

SQL> INSERT INTO Review
  2 VALUES (2, 2, 1002, 'The product quality is excellent.', DATE '2026-09-21');
1 row created.

SQL> INSERT INTO Review
  2 VALUES (3, 3, 1003, 'Good product with a pleasant fragrance.', DATE '2026-09-22');
1 row created.

SQL> INSERT INTO Review
  2 VALUES (4, 4, 1004, 'The product is useful and effective.', DATE '2026-09-23');
1 row created.

SQL> INSERT INTO Review
  2 VALUES (5, 5, 1005, 'Nice product and good quality.', DATE '2026-09-24');
1 row created.

SQL> INSERT INTO Review
  2 VALUES (6, 1, 1006, 'Very effective face pack.', DATE '2026-09-25');
1 row created.

SQL> INSERT INTO Review
  2 VALUES (7, 2, 1007, 'Good moisturizer for daily use.', DATE '2026-09-26');
1 row created.

SQL> INSERT INTO Review
  2 VALUES (8, 3, 1008, 'Gentle and refreshing cleanser.', DATE '2026-09-27');
1 row created.

SQL> COMMIT;
Commit complete.
```

```
SQL> SELECT Product_ID, Product_Name
  2 FROM Product
  3 ORDER BY Product_ID;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
1001
Aloe Vera Face Wash
1002
Rose Face Cream
1003
Coconut Hair Oil
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
1004
Herbal Body Wash
1005
Natural Lip Balm
1006
Neem Face Pack
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
1007
Cucumber Moisturizer
1008
Natural Face Cleanser

8 rows selected.
```

6. Inserting Rating Details

```
SQL> INSERT INTO Rating
  2 VALUES (1, 1, 1001, 5, DATE '2026-09-20');

1 row created.

SQL> INSERT INTO Rating
  2 VALUES (2, 2, 1002, 4, DATE '2026-09-21');

1 row created.

SQL> INSERT INTO Rating
  2 VALUES (3, 3, 1003, 5, DATE '2026-09-22');

1 row created.

SQL> INSERT INTO Rating
  2 VALUES (4, 4, 1004, 4, DATE '2026-09-23');

1 row created.

SQL> INSERT INTO Rating
  2 VALUES (5, 5, 1005, 3, DATE '2026-09-24');

1 row created.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM Rating;

RATING_ID CUSTOMER_ID PRODUCT_ID RATING_VALUE RATING_DA
-----
1          1          1001             5 20-SEP-26
2          2          1002             4 21-SEP-26
3          3          1003             5 22-SEP-26
4          4          1004             4 23-SEP-26
5          5          1005             3 24-SEP-26
```

7. Retrieve Product Review Details

```
SQL> SELECT
  2 p.Product_ID,
  3 p.Product_Name,
  4 r.Review_ID,
  5 c.Customer_ID,
  6 r.Review_Text,
  7 r.Review_Rate
  8 FROM Product p
  9 JOIN Review r
 10 ON p.Product_ID = r.Product_ID
 11 ORDER BY p.Product_ID;

PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_RATE
-----
1001
Also Very Face Wash
1
1
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_RATE
-----
Very good product and suitable for my skin.
20-SEP-26
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_RATE
-----
1002
Face Face Cream
2
2
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_RATE
-----
The product quality is excellent.
21-SEP-26
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_RATE
-----
1003
Coscent Hair Oil
3
3
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_RATE
-----
Good product with a pleasant fragrance.
22-SEP-26
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
1888
Herbal Body Wash
4 4
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
The product is useful and effective.
21-SEP-26
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
1889
Natural Lip Balm
5 5
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
Nice product and good quality.
24-SEP-26
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
1894
Scent Face Pack
6 1
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
Very effective face pack.
29-SEP-26
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
1897
```

```
Cucumber Moisturizer
9 3
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
Good moisturizer for daily use.
26-SEP-26
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
1896
Natural Face Cleanser
8 3
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_ID CUSTOMER_ID
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
Gentle and refreshing cleanser.
27-SEP-26
8 rows selected.
```

8. Calculate Average Product Ratings

```
SQL> SELECT
  2   Product_ID,
  3   ROUND(AVG(Rating_Value), 2) AS Average_Rating
  4 FROM Rating
  5 GROUP BY Product_ID
  6 ORDER BY Average_Rating DESC;

PRODUCT_ID AVERAGE_RATING
-----
1001        5
1003        5
1004        4
1002        4
1005        3
```

8.1 Identify Highly Rated Products

```
SQL> SELECT
  2   p.Product_ID,
  3   p.Product_Name,
  4   ROUND(AVG(r.Rating_Value), 2) AS Average_Rating
  5 FROM Product p
  6 JOIN Rating r
  7   ON p.Product_ID = r.Product_ID
  8 GROUP BY p.Product_ID, p.Product_Name
  9 HAVING AVG(r.Rating_Value) >= 4
 10 ORDER BY Average_Rating DESC;

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
1003
Coconut Hair Oil
5

1001
Aloe Vera Face Wash
5

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
1004
Herbal Body Wash
4

1002
Rose Face Cream

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
4
```

9. Conclusion

The Product Review and Rating Management System provides an efficient way to manage customer reviews and ratings for products. It allows customers to submit their feedback and ratings while maintaining the review information in a structured database. The system helps store, retrieve, and manage customer feedback effectively. It also helps identify customer opinions about products and supports better product evaluation. Overall, this system demonstrates how a DBMS can be used to manage product reviews and ratings accurately while maintaining data consistency and integrity.